SOFTWARE ENGINEERING • LAB 5 PRACTICAL SESSION

Designing Your Project's Architecture

PROJECT ARCHITECTURE & DESIGN SPECIFICATION

AI Sales Manager

Autonomous Social Commerce Sales Engine & Human Sales Desk (Instagram / Messenger)

TEAM MEMBER	TEAM MEMBER	TEAM MEMBER
Aymen Amara	**Rayen Oueslati**	**Moutaz Sassi**
Lead Software Architect & Backend Lead	Data & Domain Modeler & Database Specialist	Interaction Designer & AI Specialist

Table of Contents

1. Purpose of This Session
2. Session Logistics & Pre-Session Checklist
3. Session Agenda & Step-by-Step Deliverables
   - Step 1 — Choose Methodology & Architecture for Your Project
   - Step 2 — High-Level Component Design
   - Step 3 — Class Modeling
   - Step 4 — Sequence Diagrams
   - Step 5 — Apply Design Patterns
4. End-of-Session Submission Checklist
5. Team Contribution Breakdown

1. Purpose of This Session

Lab 4 walked through software architecture design steps using the shared textbook case study (the Campus Library Management System). This session is the practical hands-on companion: our team is applying those exact software engineering design principles to our real semester capstone project: AI Sales Manager.

AI Sales Manager is an AI-powered conversational sales employee and human sales desk tailored for merchants selling on Instagram and Facebook Messenger across Tunisia and the MENA region. Unlike a simple informational chatbot, our system runs an active sales workflow: it ingests high-frequency webhooks, queries grounded product stock and store policy via vector embeddings (pgvector), creates transactional order drafts in Tunisian Dinar (DT), reserves inventory, and smoothly escalates to human sales agents when complex negotiations or cash-on-delivery (COD) details occur.

> **[Key Difference from Lab 4]** There is no generic textbook case study in this document. Every architectural trade-off, component boundary, class model, sequence flow, and design pattern justification is grounded directly in our project's completed System Requirements Specification (SRS v1.2, covering FR-AUTH, FR-CAT, FR-CONV, FR-AI, FR-HAND, and FR-ORD). Every design decision traces back to concrete operational constraints.

2. Session Logistics

Item	Details
Team size	3 students: Aymen Amara, Rayen Oueslati, Moutaz Sassi
Duration	≈ 2 hours 30 minutes, timeboxed per step across all steps
Materials used	Team laptops, completed SRS v1.2, Graphviz dot, draw.io, whiteboard for live team diagramming
Instructor role	Circulates, answers questions, and signs off at checkpoints

2.1 Pre-Session Checklist

Our team completed all preparatory tasks prior to entering the lab:

- **[X] Bring completed SRS:** Completed SRS v1.2 brought to the lab. Contains formal functional specifications for Identity & Workspaces (FR-AUTH, FR-BIZ), Catalog & Variants (FR-CAT), Social Integrations & Inbox (FR-INT, FR-CONV), AI Sales Agent Autonomy & Guardrails (FR-AI), Human Handoff Rules (FR-HAND), and Order Checkout (FR-ORD).
- **[X] Re-read Lab 4 lecture refresher:** Re-read the Lab 4 architectural styles guide (Layered, Microservices, Event-Driven, Modular Monolith) and the GoF design patterns cheat sheet (Adapter, Strategy, Observer, Repository).
- **[X] Diagramming tool ready:** Software diagramming environment configured using Graphviz dot, draw.io, and shared visual whiteboard for collaborative sketching.
- **[X] Agreed on target platform/stack:** Full TypeScript stack agreed upon: NestJS 11 + Fastify on the backend, Next.js 16 (React 19) for the sales desk UI, PostgreSQL 17 + pgvector (Prisma 7 ORM) for relational and semantic persistence, and Redis 7 + BullMQ for asynchronous queue buffering.

3. Session Agenda & Step-by-Step Deliverables

Time	Step	Activity
0:00 – 0:10	**Kickoff**	Confirm team, project scope (social commerce sales assistant), and design goals with instructor
0:10 – 0:30	**Step 1**	Choose methodology (Agile/Scrum) and architectural style (Modular Monolith with async queues)
0:30 – 1:00	**Step 2**	High-level component design, component diagram & responsibility matrix
1:00 – 1:30	**Step 3**	Domain class modeling, UML class diagram & class design table
1:30 – 1:55	**Step 4**	Sequence diagrams for key use cases (Inbound AI Sales Turn & Human Order Takeover)

1:55 – 2:10	**Step 5**	Apply design patterns (Adapter, Strategy, Observer, Repository)

> **[Checkpoints]** At the end of each step, raise your hand or flag the instructor before moving to the next step. This keeps every team calibrated and catches issues early.

Step 1 — Choose Methodology & Architecture for Your Project (20 min)

Goal: Reminder: Pick the methodology and architectural style that best fit your domain and team tech stack.

In-Session Team Discussion

During the kickoff discussion, we analyzed the unique operational profile of our project:

- **Data characteristics:** Is our system data-heavy? Yes. It maintains a multi-tenant product catalog with variant SKUs, store policies, customer CRM profiles, full conversational history, and 768-dimensional vector embeddings for semantic retrieval via pgvector.
- **Real-time requirements:** Is our system real-time? Yes, on the operator side. Human sales agents require instant bi-directional messaging, live typing indicators, and real-time thread presence indicators over WebSockets (Socket.IO).
- **Asynchronous boundary:** Is our system asynchronous / event-driven? Absolutely. Meta webhook ingestion must acknowledge HTTP 200 within 50 milliseconds to prevent duplicate webhook delivery. Meanwhile, LLM reasoning and vector searches take 1.5–3.0 seconds, requiring an asynchronous queue boundary (BullMQ + Redis) to decouple webhooks from AI inference.

Methodology & Architecture Decision

We compared three architectural styles from the Lab 4 cheat sheet against our project requirements:

Option Considered	Pros for Your Project	Cons for Your Project	Chosen?
Microservices Architecture	• Independent scaling of AI worker vs. web API • Isolated failure domain for webhook ingestion • Autonomous deployment pipelines per service	• Excessive operational complexity for a 3-student team • Network latency between services compounds LLM response delay • Distributed transaction complexity (2PC/Sagas) for inventory reservation and order checkout • Complicated local dev & distributed tracing	NO
Traditional 3-Tier Layered Monolith	• Extremely rapid initial setup and low learning curve • Simple monolithic deployment • Single relational database schema with global access	• Controllers quickly degrade into spaghetti code by bypassing service boundaries • No compile-time isolation between Catalog, Messaging, and AI domains • Background AI tasks risk blocking HTTP request threads • Hard to enforce multi-tenant boundaries systematically	NO
Modular Monolith with Asynchronous Queue (NestJS + BullMQ + Redis)	• Hard compile-time domain boundaries (Catalog, Conversations, Orders, AI,	• Requires strict architectural linting rules to prevent cross-module table leakage	YES (Chosen)

	Integrations) enforced by NestJS modules • In-process method calls eliminate distributed network latency • Single PostgreSQL database guarantees ACID transactions and atomic inventory locking (SELECT … FOR UPDATE) • BullMQ queues isolate long-running AI turns from sub-50ms Meta webhook acknowledgments • Clean extraction path to microservices later if scaling requires	• Shared server memory between REST API and Socket.IO gateway unless split by process profile

Methodology Selection

We agreed as a team on Agile / Scrum with 2-week sprints. Backlog tasks are broken into atomic, user-story-driven cards with automated Test-Driven Development (TDD) and continuous quality gates (type checking, linting, architecture dependency checks, and unit tests).

Justification (2–3 sentences)

We selected a Modular Monolith architecture implemented with NestJS domain modules and an asynchronous Redis/BullMQ worker queue. This architecture provides our 3-person team with rapid development velocity, unified TypeScript type safety, and single-database ACID guarantees for inventory reservation without the distributed transaction and deployment overhead of microservices, while strictly preserving domain boundaries through encapsulated application services. Crucially, the asynchronous queue decouples slow LLM reasoning from Meta webhooks, allowing the system to achieve sub-50ms HTTP acknowledgments while keeping local development and debugging straightforward.

☐ Checkpoint: explain your choice to the instructor in one minute

Step 2 — High-Level Component Design (30 min)

Goal: Reminder: Break your system into components, each with a single clear responsibility.

System Component Decomposition

Our system architecture is decomposed into seven primary components across clear horizontal layers and vertical domain boundaries:

- **1. Web Client:** Presentation Layer (Next.js 16 Web Application): The human sales desk interface built with React 19 and Tailwind CSS. Provides the real-time unified inbox, chat timeline, manual take-over controls, product catalog drawer, and quick order draft generation.
- **2. API Gateway:** Platform & Security Kernel (NestJS Fastify Core): Manages HTTP routing, TenantGuard multi-tenant isolation, session validation, and HMAC-SHA256 signature verification for external webhooks.
- **3. Realtime Gateway:** Realtime WebSocket Gateway (Socket.IO): Maintains bi-directional WebSocket connections to operator browsers, manages room occupancy (conversation:{id}), and broadcasts instant message updates and handoff alarms.
- **4. Domain Modules:** Encapsulated Domain Modules (NestJS Backend): Domain modules with strict boundary rules (Integrations, Conversations, AI Intelligence, Catalog, Orders, Knowledge Base, and Analytics).
- **5. Background Workers:** Asynchronous Queue & Worker Subsystem (BullMQ & Redis 7): Ingests inbound messages into an asynchronous queue, manages token-fenced distributed locks (conversation:{id}:processing), and handles retry queues.
- **6. Database Layer:** Persistence & Vector Store (PostgreSQL 17 + pgvector): Stores multi-tenant relational data with ACID transaction guarantees and HNSW vector index for semantic similarity search over products and business policies.
- **7. External Services:** External Cloud Integrations: Meta Webhooks & Graph API for Instagram DM / Messenger, and LLM APIs (Google Gemini 2.5 Flash / OpenAI) for structured sales reasoning.

High-Level Component Architecture Diagram

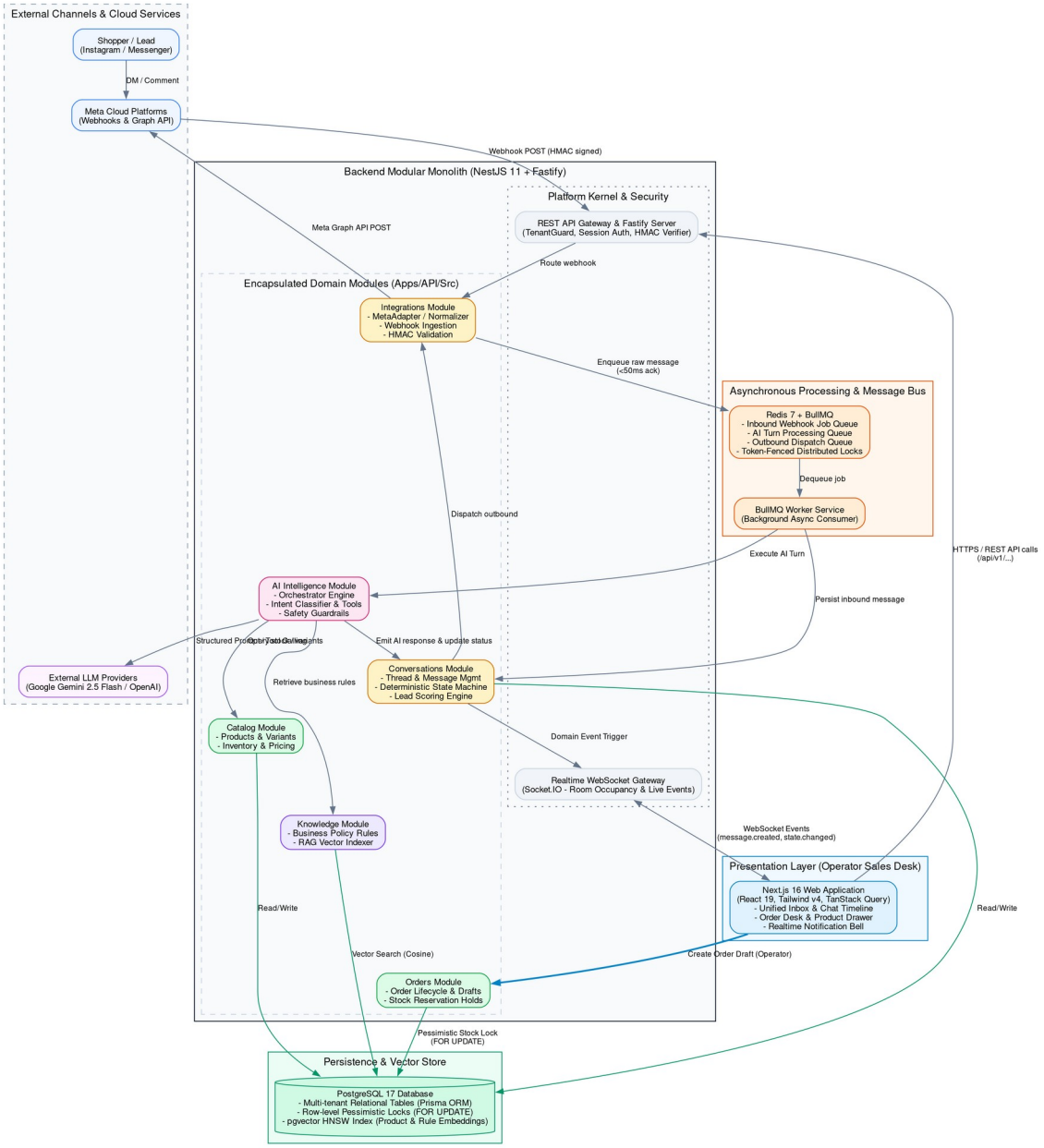


Figure 1: High-Level Component Architecture Diagram of the AI Sales Manager Platform

Component Responsibility Table

Component	Responsibility	Talks To
Next.js 16 Web Client (Presentation Layer)	Provides the operator sales desk, unified multi-channel chat timeline, manual take-over controls, product drawer, and order draft creation UI.	API Gateway (REST / HTTPS), Realtime Gateway (WebSockets / Socket.IO)
API Gateway & Platform Kernel (NestJS + Fastify)	Handles HTTP request routing, Fastify server configuration, TenantGuard multi-tenant	Next.js Web Client, All Domain Modules, Redis

	isolation, JWT session validation, and HMAC signature verification.	(Rate-limiting & session cache)
Realtime Gateway (Socket.IO Server)	Manages WebSocket connections for operator dashboard, room occupancy (conversation:{id}), and pushes instant message notifications and handoff alerts.	Next.js Web Client, Conversations Module, Orders Module
Integrations Module (MetaAdapter & Ingestion)	Ingests raw Meta webhooks (Instagram/Messenger), validates SHA-256 HMAC signatures, normalizes payloads to canonical internal events, and executes outbound Meta Graph API requests.	API Gateway, BullMQ Queue, Meta Graph API, PostgreSQL (Raw event log)
Asynchronous Worker Subsystem (BullMQ & Redis 7)	Buffers incoming message jobs, enforces token-fenced distributed locks (conversation:{id}:processing), executes background AI inference tasks, and manages retry queues.	Integrations Module, AI Orchestrator, PostgreSQL DB
AI Intelligence Module (Orchestrator & Guardrails)	Multi-turn conversational sales engine: classifies customer intent, executes tool calls (stock check, policy query), generates Darija/Arabic/French replies via LLMs (Gemini Flash), and validates safety guardrails.	Knowledge Module, Catalog Module, LLM APIs (Gemini/OpenAI), Conversations Module
Catalog & Orders Modules (Commerce Engine)	Manages product inventory, variant stock counts, atomic stock reservation holds via row-level locking (SELECT ... FOR UPDATE), and cash-on-delivery (COD) order drafts.	PostgreSQL DB, Conversations Module, Next.js Web Client

☐ Checkpoint: show your diagram and table to the instructor

Step 3 — Class Modeling (30 min)

Goal: Reminder: Identify classes, attributes, and operations from your requirements and use cases.

Domain Modeling from Requirements

From our SRS analysis, we extracted the core business entities representing the social commerce sales desk:

- **Tenant Isolation:** Multi-Tenancy Root: Workspace represents the tenant boundary (merchant business). All core domain models contain a mandatory workspaceId foreign key to enforce tenant isolation.
- **Messaging Aggregate:** Conversational Core: Customer, Conversation, and Message form the communication aggregate. A Conversation models the customer lifecycle state machine (ACTIVE_AI, NEEDS_ATTENTION, HUMAN_TAKEOVER, ORDER_PENDING, CLOSED) and tracks dynamic lead scoring.
- **Commerce Aggregate:** Commerce & Inventory: Product, ProductVariant, Order, and OrderItem represent the e-commerce transaction. ProductVariant encapsulates inventory quantities and atomic reservation logic.
- **Intelligence & Audit:** AI & Knowledge: KnowledgeDocument stores business FAQs and policies chunked for vector search, while AiDecisionTrace records structured telemetry for auditability and compliance.

UML Domain Class Diagram

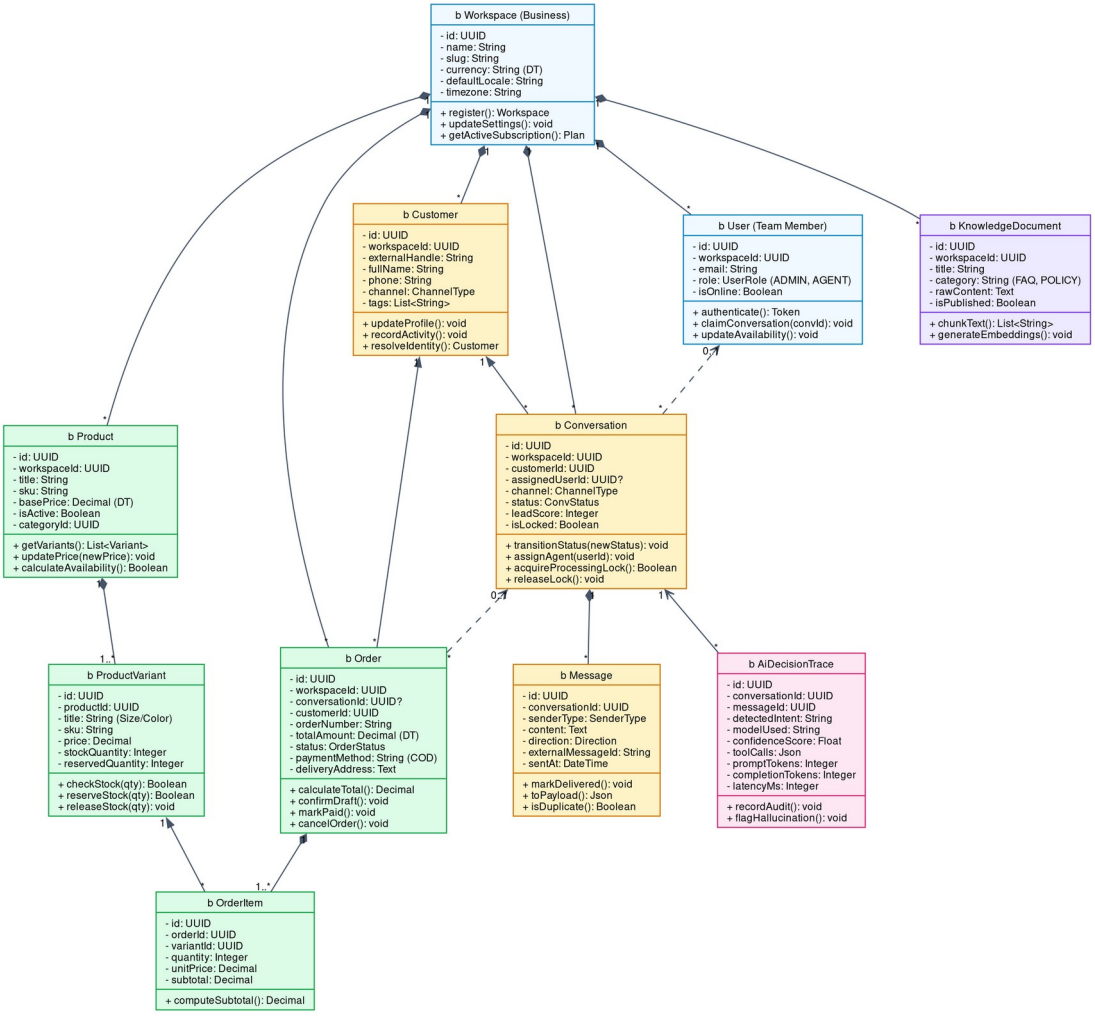


Figure 2: UML Domain Class Diagram with Attributes, Operations, and Structural Relationships

Class Design Table

Class	Attributes	Operations	Related Classes
Workspace (Business Tenant)	- id: UUID - name: String - slug: String - currency: String (DT) - defaultLocale: String - timezone: String	+ register(): Workspace + updateSettings(dto): void + getActiveSubscription(): Plan	1..* User, 1..* Customer, 1..* Product, 1..* Conversation, 1..* Order
Customer	- id: UUID - workspaceId: UUID - externalHandle: String - fullName: String - phone: String - channel: ChannelType - tags: List<String>	+ updateProfile(dto): void + recordActivity(): void + resolveIdentity(handle): Customer	1 Workspace, 1..* Conversation, 0..* Order

Conversation	- id: UUID - workspaceId: UUID - customerId: UUID - assignedUserId: UUID? - channel: ChannelType - status: ConvStatus - leadScore: Integer - isLocked: Boolean	+ transitionStatus(newStatus): void + assignAgent(userId): void + acquireProcessingLock(): Boolean + releaseLock(): void	1 Workspace, 1 Customer, 0..1 User (Agent), 1..* Message, 0..* Order, 0..* AiDecisionTrace
Message	- id: UUID - conversationId: UUID - senderType: SenderType - content: Text - direction: Direction - externalMessageId: String - sentAt: DateTime	+ markDelivered(): void + toPayload(): Json + isDuplicate(externalId): Boolean	1 Conversation, 0..1 AiDecisionTrace
ProductVariant	- id: UUID - productId: UUID - title: String (Size/Color) - sku: String - price: Decimal (DT) - stockQuantity: Integer - reservedQuantity: Integer	+ checkStock(qty): Boolean + reserveStock(qty): Boolean + releaseStock(qty): void + calculateAvailability(): Boolean	1 Product, 0..* OrderItem, 0..* StockReservation
Order	- id: UUID - workspaceId: UUID - conversationId: UUID? - customerId: UUID - orderNumber: String - totalAmount: Decimal (DT) - status: OrderStatus - paymentMethod: String (COD)	+ calculateTotal(): Decimal + confirmDraft(): void + markPaid(): void + cancelOrder(): void	1 Workspace, 1 Customer, 0..1 Conversation, 1..* OrderItem
AiDecisionTrace	- id: UUID - conversationId: UUID - messageId: UUID - detectedIntent: String - modelUsed: String - confidenceScore: Float - toolCalls: Json - latencyMs: Integer	+ recordAudit(): void + flagHallucination(): void	1 Conversation, 1 Message

☐ Checkpoint: instructor reviews at least 2 classes for cohesion and single responsibility

Step 4 — Sequence Diagrams (25 min)

Goal: Reminder: Sequence diagrams show how objects collaborate, in order, to complete a use case.

Use Case 1: Inbound Social Message Ingestion & Autonomous AI Sales Response

Requirements: Traceability: FR-INT-01 (Webhook verification), FR-CONV-01 (Conversation threading), FR-AI-01 (Grounded sales response).

Actors/Objects Involved: Customer (Instagram Shopper), Meta Webhook Service, Integrations Controller, BullMQ Queue, BullMQ Worker, AI Orchestrator, PostgreSQL (pgvector & Catalog), Google Gemini Flash LLM API, Operator Sales Desk (Socket.IO)

- **1. Customer -> Meta Platform :** Customer sends Instagram DM: 'Salam, 3andkom l'article hedha fi noir taille L?'

- **2. Meta Platform -> Integrations Controller :** Meta sends webhook payload to POST /api/v1/integrations/webhooks/meta with SHA-256 HMAC header.
- **3. Integrations Controller -> Integrations Controller :** Controller validates HMAC signature using workspace webhook secret and logs raw event to IntegrationEvent table.
- **4. Integrations Controller -> BullMQ Queue :** Controller pushes process_inbound_message job into BullMQ Redis queue.
- **5. Integrations Controller -> Meta Platform :** Controller returns HTTP 200 OK to Meta in < 50ms to prevent timeout.
- **6. BullMQ Queue -> BullMQ Worker :** Worker pops job from queue and claims message processing task.
- **7. BullMQ Worker -> PostgreSQL DB :** Worker idempotently inserts message record and acquires token-fenced lock conversation:{id}:processing in Redis.
- **8. BullMQ Worker -> AI Orchestrator :** Worker invokes AI turn orchestrator with conversation context.
- **9. AI Orchestrator -> PostgreSQL DB :** Orchestrator queries pgvector for store policy and checks CatalogService for variant availability.
- **10. PostgreSQL DB -> AI Orchestrator :** Database returns variant details: Size L, Color Black, Stock: 4 units, Price: 65 DT.
- **11. AI Orchestrator -> Gemini Flash API :** Orchestrator builds structured prompt (Tunisian Darija/French) and calls Gemini 2.5 Flash API.
- **12. Gemini Flash API -> AI Orchestrator :** LLM returns structured response with product suggestion payload and high confidence score.
- **13. AI Orchestrator -> AI Orchestrator :** Orchestrator checks guardrails: verifies price matches database exactly (65 DT) and ensures tone compliance.
- **14. AI Orchestrator -> PostgreSQL DB :** Orchestrator persists outbound AI message and AiDecisionTrace record.
- **15. AI Orchestrator -> Meta Platform -> Customer :** Integrations module sends message via Meta Graph API; customer receives DM: 'Ey marhba bik! Mawjoud fi noir taille L b 65 DT...'
- **16. BullMQ Worker -> Operator Sales Desk :** Worker emits message.created via Socket.IO gateway, updating operator dashboard in real-time.

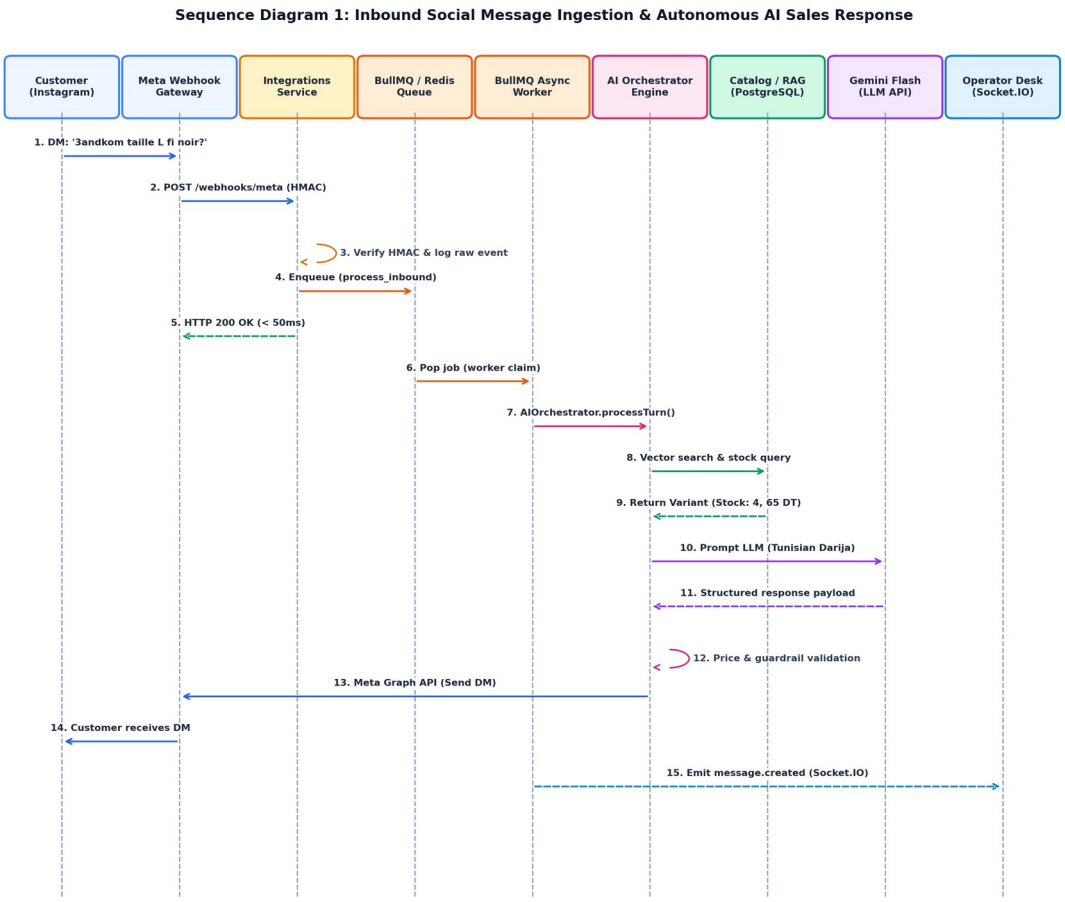


Figure 3: Sequence Diagram for Inbound Social Message Ingestion & Autonomous AI Sales Response

Use Case 2: Human Operator Takeover & Order Draft Confirmation

Requirements: Traceability: FR-HAND-01 (Discount threshold trigger), FR-ORD-01 (Order draft creation & stock reservation).

Actors/Objects Involved: Customer, Meta Cloud, AI Handoff Evaluator, Conversation State Machine, Realtime Gateway (Socket.IO), Human Operator (Next.js UI), Orders Service, PostgreSQL DB (Row-level Lock)

- **1. Customer -> Meta Cloud :** Customer sends negotiation message: 'Tnajjem ta3melli remise 3la 2 pièces w livraison l Sousse?'
- **2. Meta Cloud -> AI Handoff Evaluator :** Meta forwards webhook to worker; Evaluator analyzes message.
- **3. AI Handoff Evaluator -> AI Handoff Evaluator :** Evaluator detects custom discount negotiation request exceeding AI autonomy threshold (> 0% discount rule).
- **4. AI Handoff Evaluator -> State Machine :** Evaluator invokes triggerHandoff(convId, REASON_DISCOUNT).
- **5. State Machine -> State Machine :** State Machine transitions conversation status from ACTIVE_AI to NEEDS_ATTENTION.
- **6. State Machine -> Realtime Gateway :** State Machine emits domain event thread:needs_attention via Socket.IO gateway.
- **7. Realtime Gateway -> Human Operator :** Operator sales desk displays urgent pulsing badge and plays audible notification chime.
- **8. Human Operator -> State Machine :** Operator clicks 'Take Over' button, claiming conversation thread (POST /api/v1/conversations/:id/assign).
- **9. State Machine -> State Machine :** Status transitions to HUMAN_TAKEOVER; AI automated responses are paused.
- **10. Human Operator -> Meta Cloud -> Customer :** Operator manually types deal: 'D'accord! 120 DT les 2 pièces avec livraison offerte.'
- **11. Customer -> Meta Cloud -> Human Operator :** Customer accepts: 'Ok d'accord, ab3athli commande.'
- **12. Human Operator -> Orders Service :** Operator clicks 'Create Order Draft' button in side drawer (POST /api/v1/orders/draft).
- **13. Orders Service -> PostgreSQL DB :** Orders Service begins database transaction and executes SELECT * FROM product_variants WHERE id IN (...) FOR UPDATE.
- **14. PostgreSQL DB -> Orders Service :** Variant rows locked: database confirms stock availability (stock >= 2).
- **15. Orders Service -> PostgreSQL DB :** Service inserts 15-minute StockReservation hold and creates Order record (#ORD-7842, Status: DRAFT, 120 DT, COD).
- **16. Orders Service -> PostgreSQL DB :** Transaction commits successfully, releasing row locks.
- **17. Orders Service -> Human Operator & Customer :** Orders Service returns OrderDraftDto to frontend UI and posts invoice link to Instagram chat.
- **18. Orders Service -> State Machine :** State Machine transitions conversation status to ORDER_PENDING.

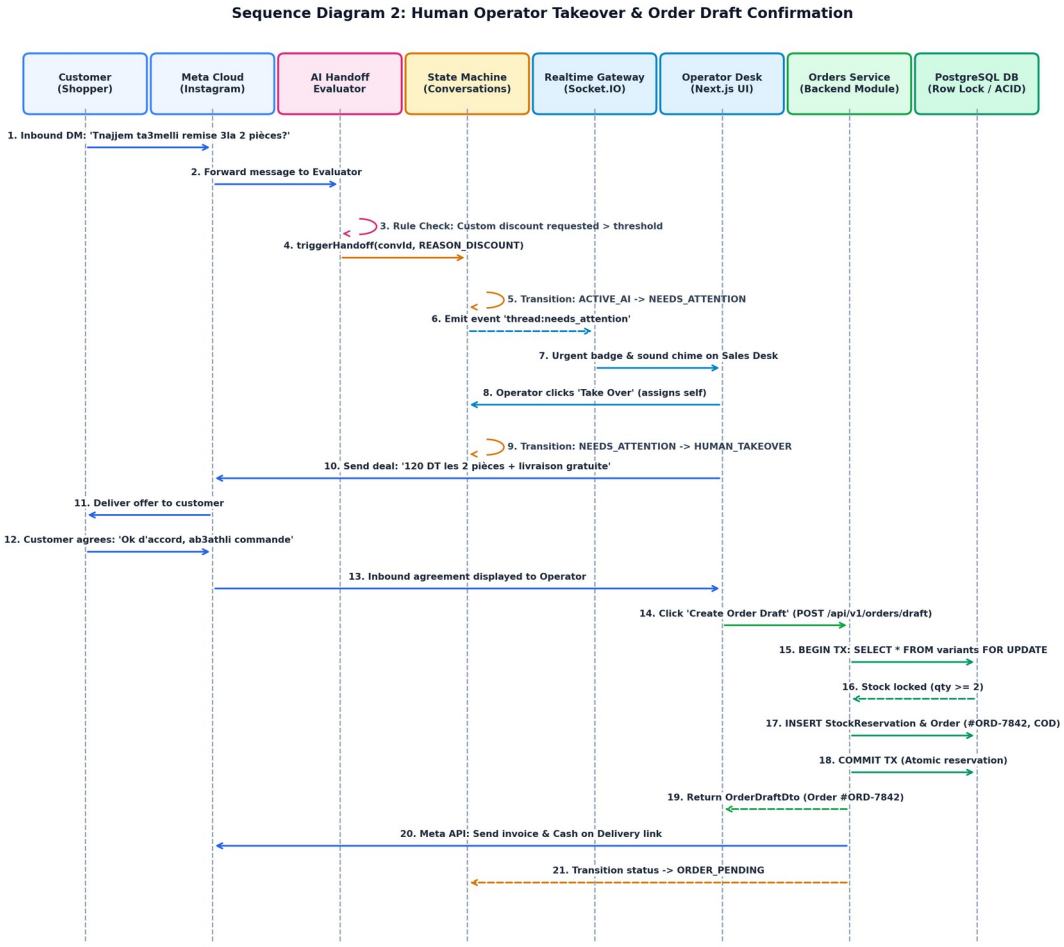


Figure 4: Sequence Diagram for Human Operator Takeover & Order Draft Confirmation

☐ Checkpoint: show your 2 sequence diagrams to the instructor

Step 5 — Apply Design Patterns (15 min)

Goal: Reminder: Reuse proven design patterns rather than inventing ad-hoc solutions.

Pattern Identification & Application

Our team evaluated design challenges across external integrations, LLM orchestration, real-time event broadcasting, and transactional order management, applying four proven GoF and enterprise patterns:

Pattern	Problem Solved	Where Applied
Adapter Pattern (Structural)	Heterogeneous external social channel APIs (Instagram Graph API, Facebook Messenger, and future WhatsApp Cloud API) format payloads differently. The Adapter converts channel-specific webhooks and outbound message schemas into a unified, canonical internal domain contract (IncomingMessageDto / OutgoingMessageDto).	IntegrationsModule (MetaAdapter, InstagramAdapter, MessengerAdapter, and WhatsAppAdapter stub)

Strategy Pattern (Behavioral)	Varying conversation complexity requires different LLM reasoning models and prompt configurations (e.g. low-cost Google Gemini 2.5 Flash for routine product FAQs vs OpenAI GPT-4o-mini for complex multi-turn sales negotiation, with automatic fallback on rate-limits). The Strategy pattern allows swapping model executors at runtime without altering core orchestrator logic.	AiModule (AiGateway, RoutingStrategy, and ModelFallbackProvider)
Observer / Pub-Sub Pattern (Behavioral)	State transitions in conversations or order creations need to trigger multiple loosely coupled downstream reactions (e.g., updating operator browser UI in real time, recalculating lead scores, dispatching merchant mobile push notifications, logging audit traces) without coupling domain services together.	NestJS EventEmitter2 for in-process events (message.received, conversation.state_changed, order.created) and Redis Pub/Sub backing the Socket.IO Realtime Gateway
Repository / Unit of Work Pattern (Architectural / Data Access)	Complex multi-step database transactions (e.g. validating product stock, acquiring a pessimistic row lock FOR UPDATE, creating a timed stock reservation hold, and inserting an order record) must be atomic and decoupled from NestJS business services and Prisma ORM details.	OrdersModule (OrdersRepository, StockReservationService), CatalogModule (CatalogRepository)

Architectural Benefits of Selected Patterns

By applying the Adapter pattern, our core messaging domain remains 100% agnostic of Meta's payload schemas; adding WhatsApp in Phase 2 requires only writing a new adapter class without touching conversation state or database models. The Strategy pattern prevents vendor lock-in with LLM providers and enables cost optimization by routing 85% of routine queries to low-cost models. The Observer pattern keeps domain services cohesive: OrdersService only emits an OrderCreatedEvent without worrying about Socket.IO client frames or SMS notifications. Finally, the Repository pattern guarantees database transaction safety for inventory reservation.

4. End-of-Session Submission Checklist

- **[X] Methodology & architecture decision with justification:** Modular Monolith with Asynchronous Queue Buffer justified against Microservices and Traditional Monolith with trade-off analysis.
- **[X] Component/architecture diagram + responsibility table:** High-resolution component architecture diagram embedded along with the 7-row Component Responsibility Table.
- **[X] Class diagram + class design table:** Comprehensive UML class diagram showing 7 core domain classes, attributes, methods, and relationships, plus Class Design Table.
- **[X] 2 sequence diagrams for key use cases:** Inbound Social AI Sales Turn & Human Takeover / Order Draft Creation sequence diagrams with numbered messages and diagrams.
- **[X] Design pattern justification table:** Adapter, Strategy, Observer/Pub-Sub, and Repository patterns justified with concrete problem statements and application locations.

5. Team Contribution Breakdown

All three team members contributed actively throughout the design and modeling practical session:

- **Aymen Amara:** Lead Software Architect & Backend Lead. Led the architectural style comparison (Modular Monolith vs. Microservices), designed the asynchronous BullMQ/Redis webhook buffer, authored the Component Responsibility Table, and verified component boundaries.

- **Rayen Oueslati:** Data Modeler & Database Specialist. Modeled the UML domain classes, defined entity attributes and operations in the Class Design Table, designed the Prisma multi-tenant isolation scheme, and formulated the pessimistic row locking mechanism (SELECT … FOR UPDATE) for inventory reservations.
- **Moutaz Sassi:** Interaction Designer & AI Specialist. Constructed both UML sequence diagrams (Inbound AI Turn & Human Takeover Flow), detailed the GoF design pattern matrix, and coordinated use case mapping to product requirements.